Let's look at an example involving *Class Macros (117)*. Do you remember the attr_accessor method from *The attr_accessor() Example*, on page 116? It generates attributes for any object:

class_definitions/class_attr.rb

```
class MyClass
  attr_accessor :a
end

obj = MyClass.new
obj.a = 2
obj.a         # => 2
```

But what if you want to define an attribute on a *class* instead? You might be tempted to reopen Class and define the attribute there:

```
class MyClass; end

class Class
  attr_accessor :b
end

MyClass.b = 42
MyClass.b      # => 42
```

This works, but it adds the attribute to *all* classes. If you want an attribute that's specific to MyClass, you need a different technique. Define the attribute in the singleton class:

```
class MyClass
  class << self
    attr_accessor :c
  end
end

MyClass.c = 'It works!'
MyClass.c               # => "It works!"
```

To understand how this works, remember that an attribute is actually a pair of methods. If you define those methods in the singleton class, they become class methods, as if you'd written this:

```
def MyClass.c=(value)
  @c = value
end

def MyClass.c
  @c
end
```